

AI Agents Setup Guide

Everything a new team member needs to get Claude Code, Codex, and Pi working with this repo's skills, tools, and optimizations. Follow in order — each section builds on the previous.

What this repo is

A shared library of AI agent skills, tools, standards, and automation scripts. Your projects don't copy from it — they import from it via live @-references. When this repo is updated, your projects pick up changes automatically on the next session.

canon/	← this repo (shared library, clone once)
skills/	← wrapup, code-reviewer, security-review, ...
tools/	← ticket, handoff
standards/	← general, git
scripts/	← hook automation (handoff, wrapup trigger, pre-commit)
guides/	← this file and context-optimization.md
extensions/pi/	← Pi lifecycle extensions
your-project/	← your work repo
CLAUDE.md	← @-imports pointing into canon
AGENTS.md	← skill table for Codex and Pi
HANDOFF.md	← session context (auto-managed)

Step 1 — Clone canon

Clone the repo anywhere you like. The scripts self-locate at runtime.

```
git clone https://github.com/sunitghub/canon.git ~/Developer/canon
```

Then set a shell variable for the rest of this guide (substitute your actual clone path):

```
export SKILLS=~/Developer/canon
```

Verify:

```
ls $SKILLS/skills.sh    # should exist
```

Step 2 — Install prerequisites

RTK (optional, recommended) — filters verbose CLI output before it hits the AI's token budget. Saves 60–90% of tokens on common operations. Setup detects

whether it's installed and wires the hook automatically if so; skills work without it.

```
# macOS
```

```
brew install rtk
```

```
# WSL / Linux
```

```
cargo install rtk
```

```
rtk --version # verify
```

If `rtk gain` fails after install, you may have the wrong package (name collision on crates.io). Use `brew install rtk` — not `cargo install rtk`.

`tk` (optional) — git-native task tracker. Only needed if you use the `ticket` skill for task management. The `wrapup` quality pipeline works without it.

```
brew tap wedow/tools
```

```
brew install ticket
```

```
tk help # verify
```

Steps 3–5 — Agent setup (automated)

Claude Code only — run the lightweight init command:

```
$SKILLS/skills.sh init
```

Re-run `skills.sh init` any time you move or rename the canon folder — it rewires the hook paths in `~/.claude/settings.json` to the new location.

Claude Code + Codex + Pi — run the full setup script. It is safe to run multiple times:

```
$SKILLS/init-agent.sh
```

It will prompt you to choose an agent (or `all`), back up any existing config files before modifying them (`.bak` extension), and report what was added vs already present.

You can also run non-interactively:

```
$SKILLS/init-agent.sh claude # Claude Code only
```

```
$SKILLS/init-agent.sh codex # Codex only
```

```
$SKILLS/init-agent.sh pi # Pi only
```

```
$SKILLS/init-agent.sh all # all three
```

What it sets up per agent:

Agent	What gets configured
Claude Code	Handoff + quality hooks merged into <code>~/.claude/settings.json</code> . RTK hook wired automatically if <code>rtk</code> is installed; skipped with a hint if not.
Codex	RTK instructions via <code>rtk init -g --codex</code> → writes <code>~/.codex/RTK.md</code> + <code>@reference</code> in <code>~/.codex/AGENTS.md</code> (skipped if RTK absent).
Pi	Copies <code>extensions/pi/handoff.ts</code> to <code>~/.pi/agent/extensions/</code>

Manual fallback (if you prefer to inspect before applying):

Claude Code — manual hook setup

```
rtk init -g --auto-patch # RTK native hook (non-interactive)
```

Then either run `$SKILLS/skills.sh init` (recommended), or merge manually into `~/.claude/settings.json` (replace `<SKILLS>` with your actual clone path):

```
{
  "hooks": {
    "Stop": [{ "matcher": "", "hooks": [{ "type": "command", "command": "<SKILLS>" } ] },
    "UserPromptSubmit": [{ "matcher": "", "hooks": [{ "type": "command", "command": "<SKILLS>" } ] },
    "PostToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command", "command": "<SKILLS>" } ] },
    "PreToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command", "command": "<SKILLS>" } ] }
  ]
}
```

Codex — manual setup

```
rtk init -g --codex --auto-patch
```

Pi — manual setup

```
mkdir -p ~/.pi/agent/extensions
cp $SKILLS/extensions/pi/handoff.ts ~/.pi/agent/extensions/handoff.ts
# then /reload in Pi
```

Step 6 — Per-project setup

Run this once per project you want to use skills in.

Register skills

All at once (recommended for new projects):

```
cd /path/to/your-project
$SKILLS/skills.sh addall
```

Or pick individually:

```
cd /path/to/your-project

# Context and task management
$SKILLS/skills.sh add handoff
$SKILLS/skills.sh add ticket      # if using tk for task tracking

# Coding standards (applied automatically, no invocation needed)
$SKILLS/skills.sh add general
$SKILLS/skills.sh add git

# Quality pipeline (installs dependencies automatically)
$SKILLS/skills.sh add wrapup
```

addall is idempotent — safe to re-run if new skills have been added to canon since you last registered.

Verify registration

```
$SKILLS/skills.sh status      # or: skills.sh --scan /path/to/your-project
```

You should see all registered skills listed under both `CLAUDE.md` and `AGENTS.md`.

Initialize HANDOFF.md

Tell Claude or Codex: “Initialize the handoff file” — it creates `HANDOFF.md` in the project root from the template.

Verification checklist

Run through these to confirm everything is wired up correctly.

RTK (if installed)

```
rtk gain      # should show "No tracking data yet" or savings stats (not an error)
rtk git status # should run and show compact output
```

Claude Code hooks

```
cat ~/.claude/settings.json  # should contain: auto-handoff, handoff-inject, auto-polish-t
```

Codex (if RTK installed)

```
cat ~/.codex/AGENTS.md    # should contain @RTK.md reference
```

Per-project

```
$SKILLS/skills.sh status    # lists registered skills
```

Skill verification

After registering skills, confirm each one is wired up and responding correctly.

Skill	How to verify	Expected response
general	<code>skills.sh status</code>	Listed under CLAUDE.md @-imports
git	<code>skills.sh status</code>	Listed under CLAUDE.md @-imports
ticket (optional)	<code>tk ls</code> — only if using <code>tk</code>	Empty list or existing tickets (no error)
handoff	Tell Claude/Codex: “Initialize the handoff file”	HANDOFF.md created in project root
code-simplifier	Tell Claude/Codex: “Simplify the changes”	Simplification report scoped to recent changes
code-reviewer	Tell Claude/Codex: “Review my changes”	Structured report across seven dimensions
security-reviewer	Tell Claude/Codex: “Run a security review”	Findings report or explicit “nothing flagged”
wrapup	Tell Claude/Codex: “Wrapup the changes” or <code>/wrapup</code>	Runs all three steps with skip reasoning for each
pdf	Tell Claude/Codex: “Extract text from [file].pdf”	Extracted content or clear error if no PDF present

Standards skills (`general`, `git`) have no invocation — they’re applied automatically to every code change. Registration in `skills.sh status` is the only verification needed.

Day-to-Day Workflows

Ticketing with `tk` (optional)

Skip this section if you don’t use `tk` for task management — `wrapup` and all other skills work without it.

tk is a git-native task tracker. Tickets are markdown files in `.tickets/` — committed to the repo, visible in git log, and clickable in VS Code.

Both Claude and Codex read the same skill file (`tools/ticket.md`) via @-import — one in `CLAUDE.md`, one in `AGENTS.md`. No agent-specific setup needed; skill updates propagate to both on next session start.

Key commands

```
tk create "title" [-t bug|feature|task|epic|chore] [-p 0-4] [-d "desc"]
tk ls                                     # open tickets
tk ls --status=in_progress               # filter by status
tk show <id>                             # full ticket detail
tk start <id>                           # mark in_progress
tk close <id>                           # mark closed (prefer: use approve workflow)
tk reopen <id>                           # reopen a closed ticket
tk dep <id> <dep-id>                     # id depends on dep-id
tk dep tree <id>                         # visualize dependency graph
tk dep cycle                             # detect circular dependencies
```

Priority: 0 = highest, 4 = lowest. Default is 2.

Standard workflow

1. **Create** — Ask the agent: “Create a ticket to add X.” The agent runs `tk create` and returns the ticket ID.
2. **Implement** — Ask the agent: “Implement ticket <id>.” The agent runs `tk start <id>`, does the work, and prepends `<id>:` to every commit.
3. **Test** — Review the changes yourself.
4. **Approve** — Tell the agent: “Approve <id>.” The agent runs the full pipeline (see below).

If during testing you discover a dependency (something that needs to be done first):

- 4a. Tell the agent: “Create a ticket for Y and make it a dependency of <id>.” The agent runs `tk create` for Y, then `tk dep <id> <dep-id>` to link them.
- 4b. Ask the agent to implement the dependency: “Implement <dep-id>.”
- 4c. Test the dependency work, then just approve the root: “Approve <id>.” The agent walks the full dependency tree, closes leaves first, works up to the root, then runs the pipeline once across all modified files. You never need to track or list the deps yourself.

Approve pipeline Say “approve <id>” (or “ship it”, “approve and close”) after testing. The agent runs:

1. `tk dep cycle` — aborts if cycles are detected
2. `tk dep tree <id>` — walks the full tree; closes leaf dependencies first, then works up to the root
3. `tk close <id>`
4. Runs `/wrapup` on all files modified since the ticket was started

You only ever need to approve the root ticket — the agent handles the rest.

Agents are instructed never to call `tk close` directly — always through the approve pipeline so wrapup never gets skipped.

Dependency management

```
# Mark that ticket nw-05 cannot close until nw-03 is done
tk dep nw-05 nw-03
```

```
# Inspect the tree before approving
tk dep tree nw-05
```

```
# Check for cycles before closing a milestone
tk dep cycle
```

If a dependency is still open when you approve, the agent will warn you and ask whether to close the dep first or proceed anyway.

Wrapup — Quality Pipeline

Wrapup is the quality gate that closes out any unit of work. It does not require the ticket system — run it any time you finish a chunk of code.

How to trigger

`/wrapup`

Or: “Wrapup the changes” / “Wrapup the auth refactor” / “Wrapup ticket nw-42.”

Pipeline

`code-simplifier` → `code-reviewer` → `security-review`

Each step is skipped if it doesn’t apply — the agent states why in one line when skipping.

Step	Skipped when
<code>code-simplifier</code>	Single-line change, or docs/config only

Step	Skipped when
code-reviewer	Single-line fix with no design implications, or purely mechanical change
security-review	No security-sensitive files changed (auth, DB queries, user input, API endpoints, crypto, session management, file I/O, env/secret access)

What each step produces **Simplifier** — rewrites recently modified code for clarity without changing behavior: reduces nesting, eliminates redundancy, improves names, removes obvious comments. Scope is limited to files touched in the current session.

Reviewer — structured report across seven dimensions: correctness, maintainability, readability, efficiency, security, edge cases, test coverage. Format: Critical → Improvements → Nitpicks → Recommendations.

Security review — high-confidence findings only. Traces data flow end-to-end before flagging anything. Reports severity (Critical / High / Medium / Low) with location, pattern, evidence, impact, and fix.

Final output format

```
## Wrapup Report - <description of work>
```

```
### code-simplifier
- <what was simplified and where>
```

```
### code-reviewer
- [Critical] ...
- [Improvement] ...
```

```
### security-review
- [High] ...
```

Address criticals before committing. Improvements and nitpicks are at your discretion.

Auto-trigger (optional, requires ticket skill) If you use the `ticket` skill, the `PostToolUse` hook (`auto-polish-trigger.sh`) triggers wrapup automatically when a ticket is closed via the approve workflow. Without tickets, run `/wrapup` manually whenever you want to close out a chunk of work.

Staying updated

When this repo is updated with improved skills or new scripts:


```
cd $SKILLS && git pull
```

That's it for existing skills. Because your project's `CLAUDE.md` uses live @-import references into this repo, Claude Code picks up updated skill content automatically on the next session. Hook scripts update immediately too — they're called by path.

For newly added skills: opt in with `addall` (picks up everything new at once) or individually:

```
$SKILLS/skills.sh addall /path/to/your-project      # register any new skills  
$SKILLS/skills.sh add <new-skill> /path/to/your-project  # or just one
```

Check what's new:

```
$SKILLS/skills.sh list
```

How the automation works end-to-end

See `guides/context-optimization.md` for a full explanation of the token optimization, session handoff, and quality pipeline — including why each piece is designed the way it is.